Quality attribute	Requirement	Design response
Usability	Users must understand loading, errors, empty results, score context, and available actions	Role-specific pages, normalized labels and reasons, validation signals, and explicit action controls
Auditability	Significant automated or administrative events should be reconstructable	audit_log, notification delivery records, email-action state, timestamps, actor/source metadata, and administrative views
Explainability	The system should expose evidence without presenting a score as a hiring probability	Lexical term/skill reasons, separate labels and application states, policy settings, and documented limitations

Security and auditability are defense-in-depth properties. A role rule at the URL layer does not replace ownership verification in a service, and an audit row does not make an unsafe action safe. Similarly, a database index is a performance design decision, not proof of latency under load. The evaluation chapter must test each critical claim in the environment in which it is reported.

1.5 Use-Case Analysis

1.5.1 Candidate Uploads a CV and Receives Matches

Table 1.5. Use case - Candidate uploads a CV and receives matches

Field	Description
Use-case ID	UC-01
Primary actor	Candidate
Preconditions	The Candidate account is active and authenticated.
Trigger	The Candidate uploads a supported document or submits manual CV data.
Main flow	Validate the source; store the CV; extract and normalize text; derive terms and skills; create or update the vector; score eligible active Jobs; persist unique CV-Job Matching records; return ordered match cards.
Alternative/exception flows	Reject unsupported or oversized files and invalid fields; record OCR or extraction failure; allow soft quality warnings; return an empty result when no active Job is eligible.
Postconditions	The CV ends in SCORING_DONE or FAILED, with a visible status and failure reason where applicable.

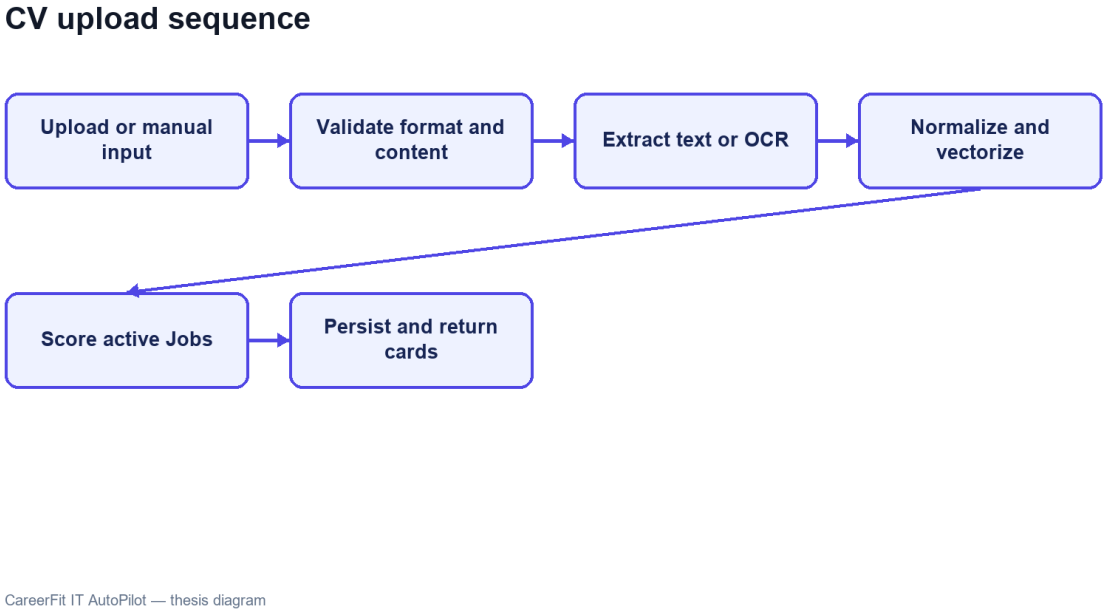

Figure 1.5. CV ingestion and matching sequence

1.5.2 Candidate Searches, Applies, and Withdraws

Table 1.6. Use case - Candidate searches, applies, and withdraws

Field	Description
Use-case ID	UC-02
Primary actor	Guest or Candidate
Preconditions	Public search needs no account; application and personalized scoring require an authenticated Candidate.
Trigger	The user searches Jobs, opens details, or selects Apply/Withdraw.
Main flow	Search and filter public Jobs; open Job details; resolve the Candidate and selected/default CV; verify the Job; prevent duplicate Candidate-Job applications; create an application; list owned applications; withdraw when the current state permits.
Alternative/exception flows	Reject missing authentication, missing/default CV, invalid Job, duplicate application, unrelated ownership, or invalid withdrawal state.
Postconditions	A valid application is created or withdrawn, and the resulting state is returned to the owning Candidate.

1.5.3 Recruiter Creates a Job and Reviews Candidates

Table 1.7. Use case - Recruiter creates a Job and reviews candidates

Field	Description
Use-case ID	UC-03
Primary actor	Recruiter
Preconditions	The Recruiter account is active and authenticated.
Trigger	The Recruiter creates a Job or opens an owned Job workspace.
Main flow	Validate structured and free-text Job data; normalize and vectorize the Job; persist ownership; expose eligible Jobs to matching; rank applicants and discovered candidates; invite a Candidate or update application state.
Alternative/exception flows	Reject invalid salary/content rules, missing ownership, another Recruiter's Job identifier, invalid invitation, or unsupported application transition.
Postconditions	The owned Job and permitted recruitment actions are persisted with audit evidence.

1.5.4 Candidate Feedback Changes Later Ranking

Table 1.8. Use case - Candidate feedback changes later ranking

Field	Description
Use-case ID	UC-04
Primary actor	Candidate
Preconditions	The Candidate owns the CV and Matching associated with the feedback.
Trigger	The Candidate submits GOOD_MATCH, POTENTIAL, BAD_MATCH, or NOT_INTERESTED feedback.
Main flow	Resolve the Matching and actor; verify ownership and role; upsert one current judgment; commit the transaction; start Rocchio learning after commit; mark affected Matchings for recomputation; rescore and clear the marker.
Alternative/exception flows	Reject another role, another Candidate's Matching, missing evidence, or invalid feedback; retain retry visibility when asynchronous recomputation fails.
Postconditions	The feedback and audit state are stored, and later rankings use the rebuilt learned vector.

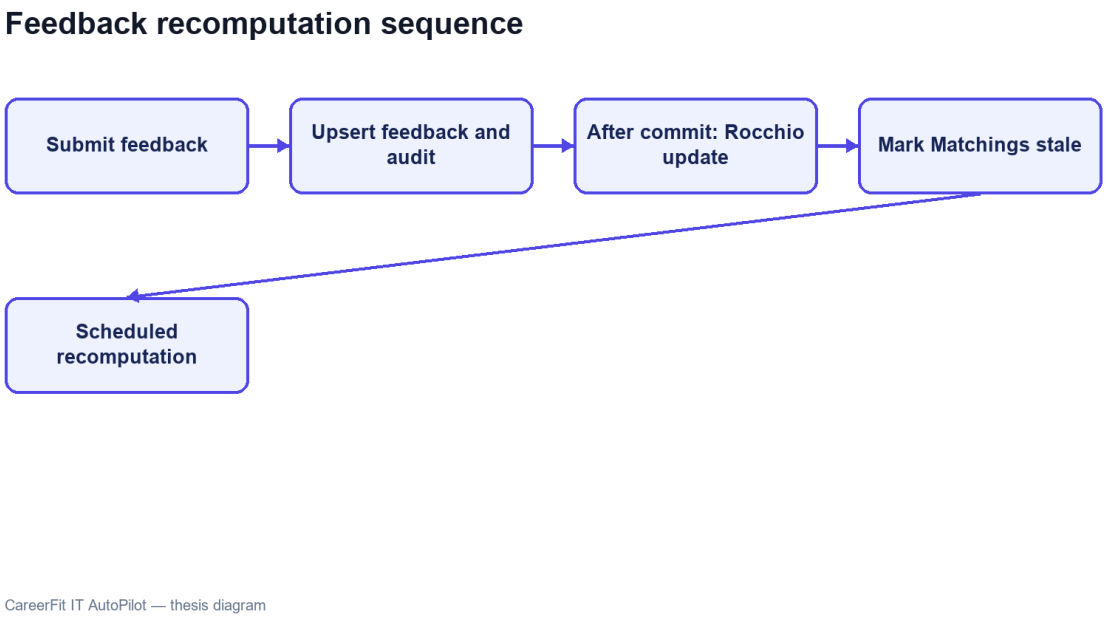

Figure 1.6. Feedback learning and recomputation sequence

1.5.5 AutoFit Executes a Policy-Governed Action

Table 1.9. Use case - AutoFit executes a policy-governed action

Field	Description
Use-case ID	UC-05
Primary actor	Candidate and background scheduler
Preconditions	An enabled owned policy, an eligible default CV, completed scoring, and an allowed action state exist.
Trigger	A scheduled scan or controlled run-now request evaluates the policy.
Main flow	Resolve the user, Candidate, CV, matches, thresholds, consent, quota, cooldown, quiet hours, time zone, and prior interactions; select eligible actions; create notifications or applications; write audit evidence.
Alternative/exception flows	Skip disabled, paused, ineligible, duplicate, quota-exceeded, cooldown, quiet-hour, stale, or invalid-state items; isolate per-item failures.
Postconditions	Only policy-allowed actions are recorded, and the remaining items stay unchanged for later evaluation.

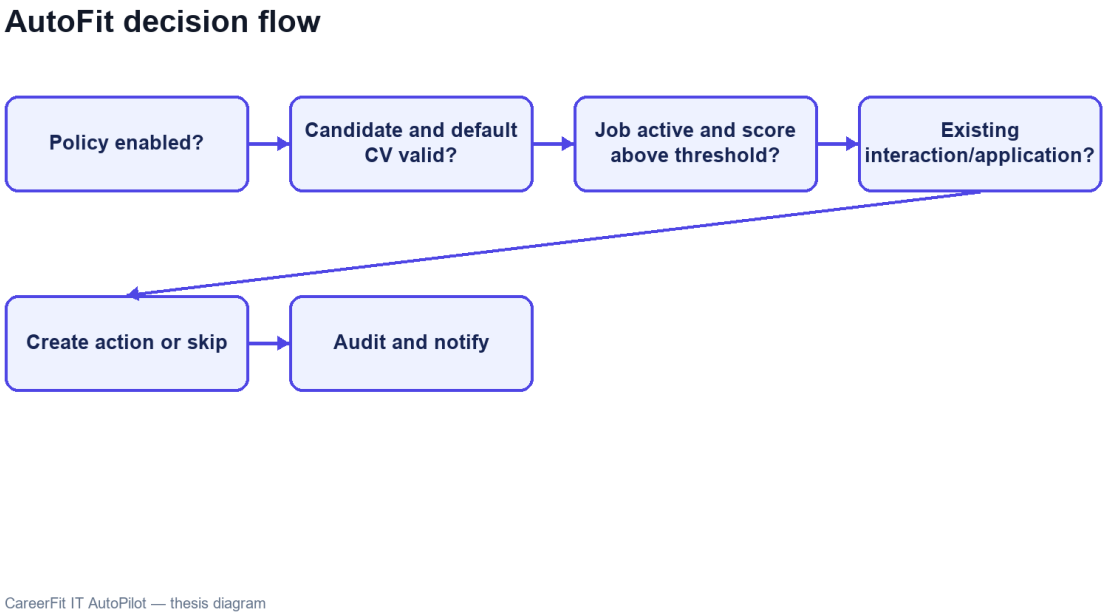

Figure 1.7. AutoFit decision flow

1.5.6 User Completes an Email Feedback Action

Table 1.10. Use case - User completes an email feedback action

Field	**Description**
Use-case ID	UC-06
Primary actor	Email recipient
Preconditions	A pending, unexpired EmailAction exists and only its SHA-256 token hash is stored.
Trigger	The recipient opens the link and deliberately confirms the action.
Main flow	GET hashes and validates the token and displays a confirmation page without changing recruitment data; POST repeats validation, performs the supported action, marks the token redeemed, and records the outcome.
Alternative/exception flows	Reject missing, expired, redeemed, malformed, or unsupported actions; mark expired pending records; do not execute state changes from GET.
Postconditions	The requested action is executed at most once under normal validation and remains auditable.

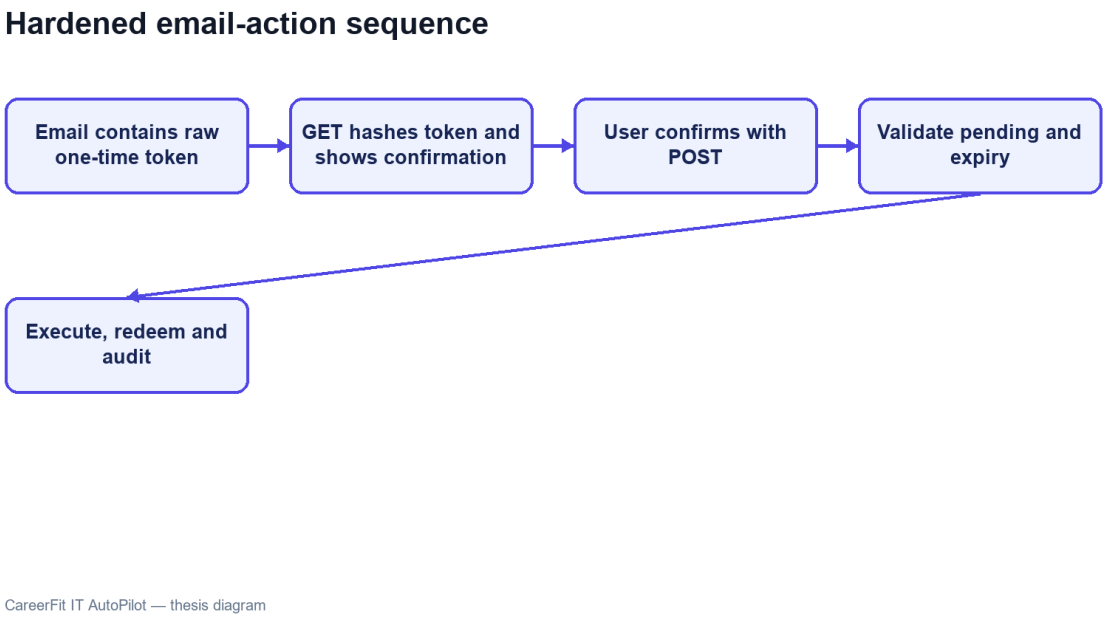

Figure 1.8. Secure email-action confirmation and redemption sequence

1.6 Chapter Summary

This chapter defined CareerFit's actors, functional and non-functional requirements, and six representative use cases. The requirements separate matching evidence, application state, user policy, automated actions, and audit records so that later design and implementation decisions can be traced to expected behavior.

CHAPTER 2. THEORETICAL BACKGROUND AND SOLUTION DESIGN

2.1 Recruitment Information and CV–Job Description Matching

Recruitment matching is an information-retrieval and decision-support problem that compares different types of evidence about candidates and vacancies. A curriculum vitae usually describes education, work history, projects, technical skills, certifications, languages, and contact information. A job description lists responsibilities, required and preferred skills, seniority, location, employment conditions, and company context. Some data is structured, such as years of experience or location, while other data is free text and may use inconsistent terms. Therefore, exact database filters are useful for fixed conditions but are not enough to rank the overall fit between a candidate and a position.

The same information supports different user tasks. From the candidate perspective, the system retrieves and ranks jobs that fit a CV or desired profile. From the recruiter perspective, it ranks applicants or discovers potential candidates for a specific vacancy. These directions share document representations and similarity functions, but they are not identical business decisions. A relevant job recommendation does not imply that an application has been submitted, and a high candidate–job similarity does not imply that a recruiter should accept the candidate. The score is therefore evidence for prioritization rather than a final employment decision.

Recruitment data also has domain-specific ambiguity. A technology may be expressed by an abbreviation, a product name, or a broader skill family; job titles vary across companies; and the same term may have different importance at junior and senior levels. Open recruitment corpora are relatively uncommon because CVs contain personal information. The Djinni Recruitment Dataset illustrates both the scale of the matching problem and the value of anonymized, documented corpora, providing more than 150,000 jobs and 230,000 candidate profiles in English and Ukrainian [6]. CareerFit is narrower: it focuses on IT recruitment and uses explicit fields and normalized text so that matching behavior can be inspected within the project scope.

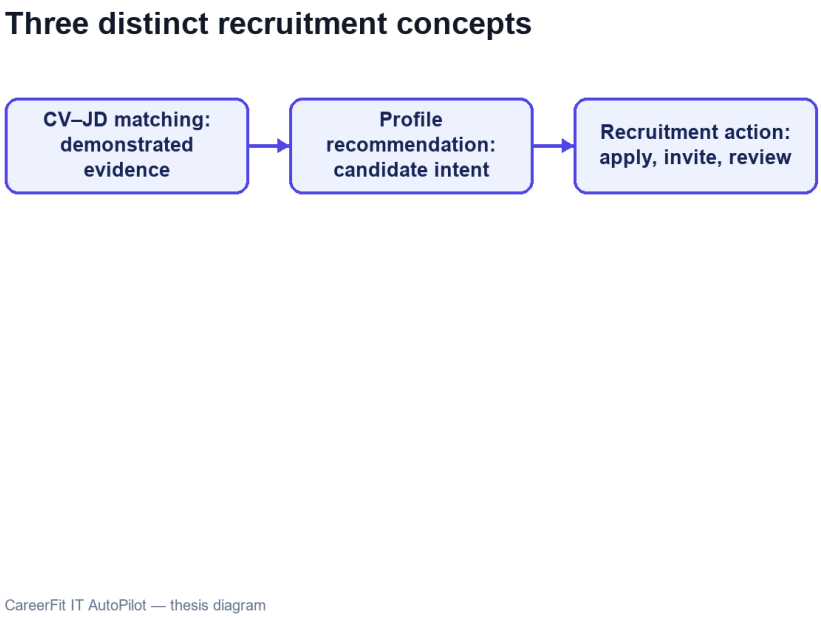

Figure 2.1. Distinction between matching, recommendation, and recruitment action

2.2 Text Representation and Preprocessing

2.2.1 Document Normalization

Before vectorization, text must be converted into a consistent form. A typical pipeline extracts machine-readable text, normalizes character encoding and letter case, separates or standardizes tokens, removes formatting noise, and may filter stop words or apply stemming or lemmatization. For recruitment documents, preprocessing must keep important technical tokens such as framework names, programming languages, versions, and abbreviations. Removing too much information can make a CV and a JD appear less related even when they share an important technology.

Bilingual data adds further complexity. Vietnamese contains diacritics and multi-syllable expressions separated by spaces, while English technical terminology frequently appears unchanged inside Vietnamese sentences. A practical pipeline must apply deterministic normalization to equivalent inputs and maintain a controlled vocabulary. It should also keep structured attributes—such as location, language, seniority, and salary mode—available for validation or filtering instead of forcing every business constraint into a single text vector.

Preprocessing quality directly affects every later score. Missing OCR text, malformed documents, extremely short descriptions, duplicated boilerplate, and contradictory structured fields can distort term frequencies. For this reason, validation belongs before scoring. Hard validation rejects inputs that cannot support meaningful processing, while soft validation allows processing but records warnings or suggestions. This distinction prevents the ranking engine from silently assigning apparently precise scores to unusable data.

2.2.2 Bag-of-Words Representation

In a bag-of-words representation, a document is represented by the terms it contains and their weights; word order is not modeled directly. If the vocabulary contains |V| terms, each document is mapped to a vector in R^|V|. This representation is sparse because a particular CV or JD contains only a small portion of the complete vocabulary. Its main advantages are computational simplicity, reproducibility, and the ability to inspect which terms contribute to similarity. Its main limitation is lexical dependence: synonyms and related skills do not match unless normalization or an explicit mapping connects them.

2.3 TF-IDF and Cosine Similarity

2.3.1 Term Frequency and Inverse Document Frequency

The vector-space model represents documents and queries as weighted term vectors and ranks documents according to their geometric relationship [1]. TF-IDF is a standard weighting family for this model [2]. Term frequency (TF) reflects how strongly a term occurs in a particular document. Inverse document frequency (IDF) reduces the influence of terms that occur in many documents and increases the relative influence of terms that distinguish a smaller subset of the corpus.

Let tf(t,d) denote the frequency of term t in document d, let N be the number of documents in the reference corpus, and let df(t) be the number of documents containing t. A basic formulation is:

$$tfidf(t,d) = tf(t,d) \times log(N / df(t)).$$

Implementations often use logarithmic TF scaling, smoothed IDF, or vector normalization to handle repeated terms and unseen values. The exact formulation changes numeric scores, so a thesis must document the implemented formula rather than referring to TF-IDF as if it were a single universal function. A controlled or static reference corpus can improve score stability because the IDF of a term does not change whenever a user adds one document; however, it may become less representative as technologies and hiring terminology evolve.

2.3.2 Cosine Similarity

Cosine similarity compares the direction of two vectors rather than their unnormalized magnitude. For vectors x and y, it is defined as:

$$cos(x,y) = (x \cdot y) / (||x||_2 \, ||y||_2).$$

For non-negative TF-IDF vectors, cosine similarity usually lies between 0 and 1. A larger value indicates a greater proportion of shared weighted terms. Length normalization is valuable for CV–JD comparison because a long CV should not receive a higher score merely because it contains more words. TF-IDF-weighted cosine distance has also been used as an efficient and reproducible document-alignment method [3].

Cosine similarity remains a lexical measure. If a JD uses “container orchestration” while a CV only uses “Kubernetes,” the vectors may not overlap unless the vocabulary or normalization layer establishes that relationship. Conversely, shared generic terms can increase similarity without proving that mandatory requirements are satisfied. CareerFit therefore treats similarity as one component of a broader workflow that also includes structured fields, validation, labels, reasons, user feedback, and policy conditions.

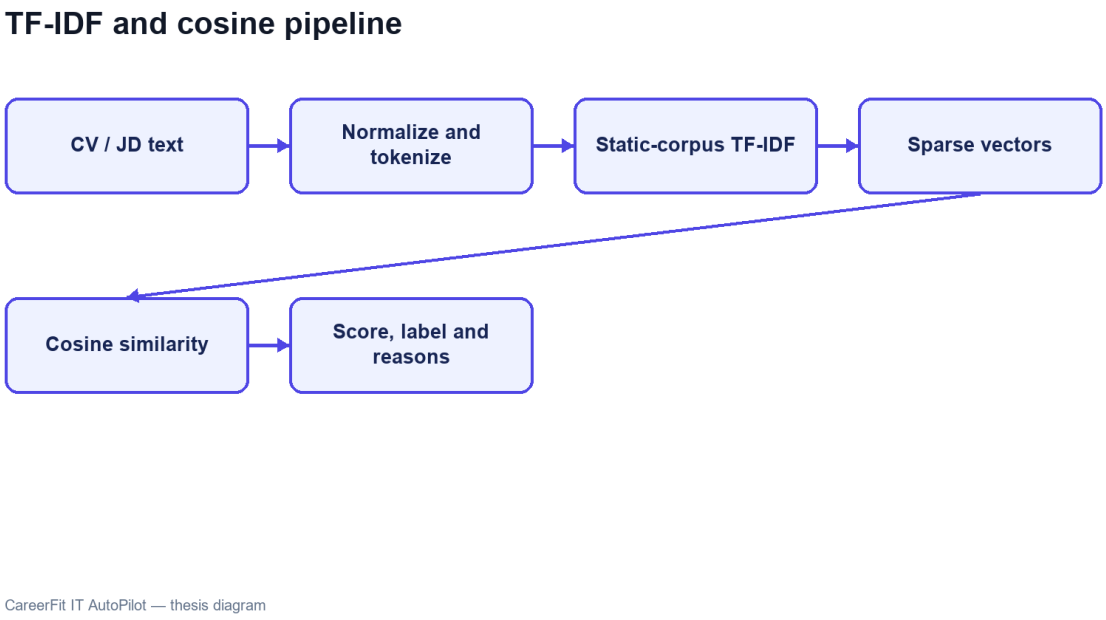

Figure 2.2. TF-IDF vectorization and cosine-similarity pipeline

2.4 Matching, Recommendation, and Ranking

Matching and recommendation can be formalized as ranking tasks. Given a query representation q and a candidate set D, the system computes a relevance score s(q,d) for each d □ D and orders the results by decreasing score. For candidate-to-job matching, q may be a CV vector and D the active job vectors. For recruiter-side discovery, q may be the selected JD and D the available candidate or CV vectors. For profile-based recommendation, q represents the candidate’s desired role, skills, location, seniority, and other preferences rather than one uploaded CV.

This separation matters because a CV describes past experience, while a preference profile describes what the candidate wants. A candidate may have Java experience but look for a Go position, so CV-based matching and preference-based recommendation can produce different but valid lists. Keeping the workflows separate also makes evaluation clearer because the labels, candidate sets, and user goals of one task should not be assumed to fit the other task.

A raw similarity value is not automatically a calibrated probability of hiring success. Mapping it to a percentage or label is a presentation and business-rule decision, not a statistical guarantee. Thresholds such as Low, Medium, High, or Potential should be documented, tested at boundary values, and kept distinct from

application status. Stable secondary sorting is also necessary when multiple items have equal scores so that repeated requests do not appear inconsistent.

Recent resume–job matching research increasingly uses dense encoders and contrastive learning. ConFit v2, for example, improves resume–job representations through hypothetical resumes and hard-negative mining [7]. Such approaches can capture semantic relations beyond exact terms, but they require training data, model governance, and an evaluation design appropriate to the domain. CareerFit intentionally begins with a transparent lexical baseline whose calculations can be reproduced and explained, while treating semantic or hybrid retrieval as future work rather than an implemented result.

2.5 Relevance Feedback and the Rocchio Method

2.5.1 Relevance Feedback

Relevance feedback uses judgments about retrieved items to modify a query or profile representation. Instead of assuming that the original text completely expresses the user's information need, the system observes which results are marked relevant or non-relevant. Explicit feedback is especially useful in recruitment because the importance of related skills may be known to the candidate or recruiter but omitted from the original CV, profile, or JD.

The classic Rocchio method updates a query vector by combining the original query with the centroids of relevant and non-relevant document vectors [4]. Let q_0 be the original vector, D_r the set of relevant feedback documents, and D_{nr} the set of non-relevant documents. A common form is:

$$q_m = \alpha q_0 + (\beta / |D_r|) \Sigma(d \in D_r)\, d - (\gamma / |D_{nr}|) \Sigma(d \in D_{nr})\, d.$$

The parameters α, β, and γ control retention of the original representation, attraction toward relevant examples, and movement away from non-relevant examples. They are hyperparameters rather than universal constants. Positive and negative feedback may also be weighted differently because an explicit rejection can be ambiguous: a user may reject a job because of location, timing, or salary even when its technical content is relevant.

2.5.2 Feedback Semantics in CareerFit

CareerFit distinguishes feedback such as GOOD_MATCH, POTENTIAL, BAD_MATCH, and NOT_INTERESTED. These labels should not be collapsed without a documented mapping. GOOD_MATCH provides a strong positive relevance signal. POTENTIAL may represent partial or transferable fit rather than direct relevance. BAD_MATCH is a negative judgment about the match. NOT_INTERESTED can reflect personal intent and may be weaker evidence about technical similarity. The implementation chapter must state exactly how each event affects the learned vector and whether role or channel changes its weight.

A reliable update process keeps the original base vector and rebuilds the learned vector from the current feedback history. Updating the previously learned vector again and again can cause cumulative drift: the same recomputation may change the output even when there is no new feedback. In CareerFit, the same base vector, feedback set, and parameters should always produce the same learned vector. This repeatable behavior is important for retries, scheduled jobs, debugging, and audits.

Feedback learning should be evaluated by comparing rankings before and after a defined feedback event in a controlled setup. The experiment records the baseline ranking, adds feedback, rebuilds the representation, and measures the effect on separate holdout items. If the same positive item is used both as feedback and as the only proof of improvement, the result is circular. Chapter 4 therefore separates feedback examples from holdout evaluation and clearly labels synthetic scenarios.

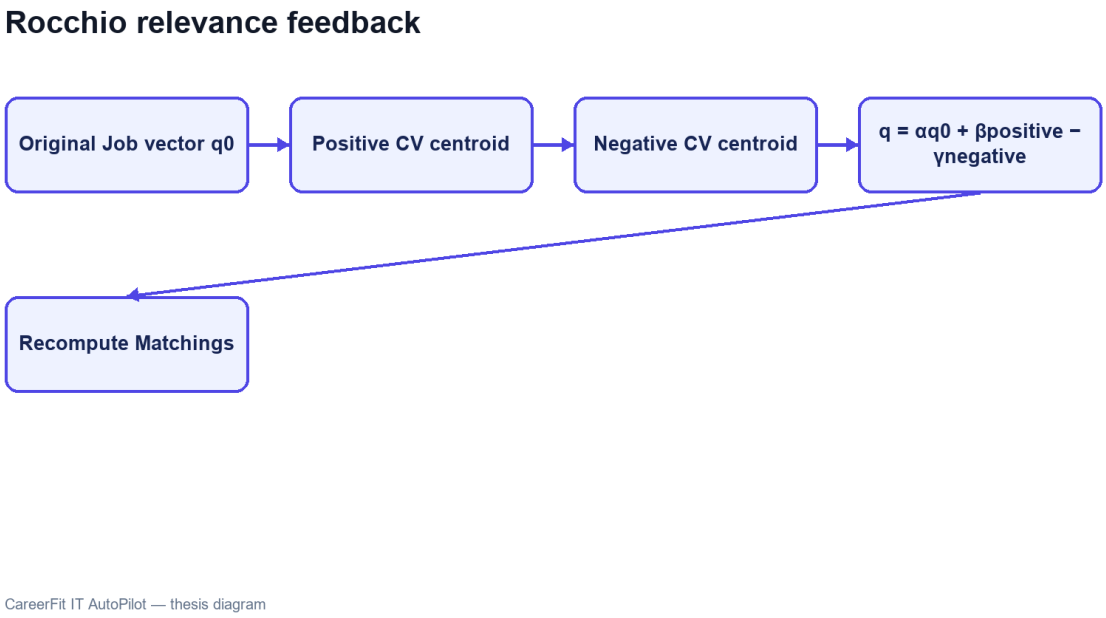

Figure 2.3. Rocchio relevance-feedback vector update

2.6 Evaluation Metrics for Ranked Results

Ranking quality cannot be described well by ordinary classification accuracy because users see an ordered list and usually inspect only the first few items. Evaluation therefore needs relevance judgments and metrics that consider the cutoff K, the positions of relevant results, and graded relevance when it is available.

2.6.1 Precision@K, Recall@K, and HitRate@K

Precision@K is the proportion of the top K results that are relevant. Recall@K is the proportion of all known relevant items that appear in the top K. HitRate@K records whether at least one relevant item appears in the first K positions. Precision emphasizes the usefulness of the visible list, recall emphasizes coverage, and HitRate provides a simple task-success indicator. The metrics answer different questions and should not be substituted for one another.

2.6.2 Mean Reciprocal Rank

For a query, reciprocal rank is 1/r, where r is the rank of the first relevant result; it is zero if no relevant result is retrieved. Mean Reciprocal Rank (MRR) averages this value across queries. MRR strongly rewards placing the first relevant result near the top but ignores the ordering of additional relevant items. It is therefore appropriate when finding one useful job or candidate is the primary objective, but it should be accompanied by a broader top-K metric when multiple relevant results matter.

2.6.3 Discounted Cumulative Gain

Discounted Cumulative Gain (DCG) supports graded relevance and discounts useful items that occur at lower ranks. Normalized DCG divides the observed DCG by the ideal ordering for the same relevance judgments, producing nDCG values that are comparable across queries. Järvelin and Kekäläinen introduced cumulated-gain measures to credit systems for ranking highly relevant documents before less relevant ones [5]. One common formulation is:

$$DCG@K = \Sigma_{i=1..K} (2^{rel_i} - 1) / \log_2(i + 1), \quad nDCG@K = DCG@K / IDCG@K.$$

Metric values are meaningful only when the relevance labels, candidate set, cutoff K, and calculation method are documented. A large improvement in a synthetic scenario shows behavior only in that scenario; it does not prove hiring quality in production. Chapter 4 therefore reports the data source, baseline, holdout logic, repeated-run behavior, and threats to validity together with the metric values.

2.7 Human-in-the-Loop and Explainable Automation

2.7.1 Levels of Human Involvement

Human-in-the-Loop is not a single interface feature. Human involvement can occur during data validation, model configuration, review of recommendations, approval of actions, exception handling, and post-action feedback. NIST describes human–AI configurations as ranging from manual decision making through systems that provide an additional opinion or defer to an expert, to more autonomous operation [9]. The appropriate configuration depends on the consequences and failure modes of the task.

Recruitment decisions can strongly affect people, so a similarity engine should help users set priorities instead of acting as the final decision-maker. CareerFit separates input processing, decision support, actions, learning, and audits. The matching engine provides evidence; the AutoFit policy checks whether an action is allowed; users can review or confirm important actions; feedback changes later rankings; and audit records support investigation. This design does not remove bias, but it makes the control points and system state clearer.

2.7.2 Explainability and Auditability

Explainability describes why a result or action was produced. Auditability asks whether the inputs, rules, actor, channel, time, and outcome can be checked later. A list of shared skills can explain lexical similarity, but it does not prove that the whole model or decision is fair. In the same way, an audit log records what happened but does not prove that the policy was correct.

Research on algorithmic hiring warns that claims about reducing bias and improving performance must be checked against real practices, data, and organizational context [10]. Explanations can help users understand a result, but they should be based on the features that the system actually uses. Research on explainable recommendation also separates scoring from the user-facing reasons shown with a result [14]. CareerFit therefore uses match reasons and policy outcomes that can be checked instead of unsupported natural-language claims about candidate suitability.

The NIST AI Risk Management Framework groups AI risk work into governance, mapping, measurement, and management [9]. CareerFit is not a complete implementation of this framework. However, its design follows several practical ideas: state the scope and limitations, keep people responsible for important actions, measure behavior with defined evidence, and keep records for later review.

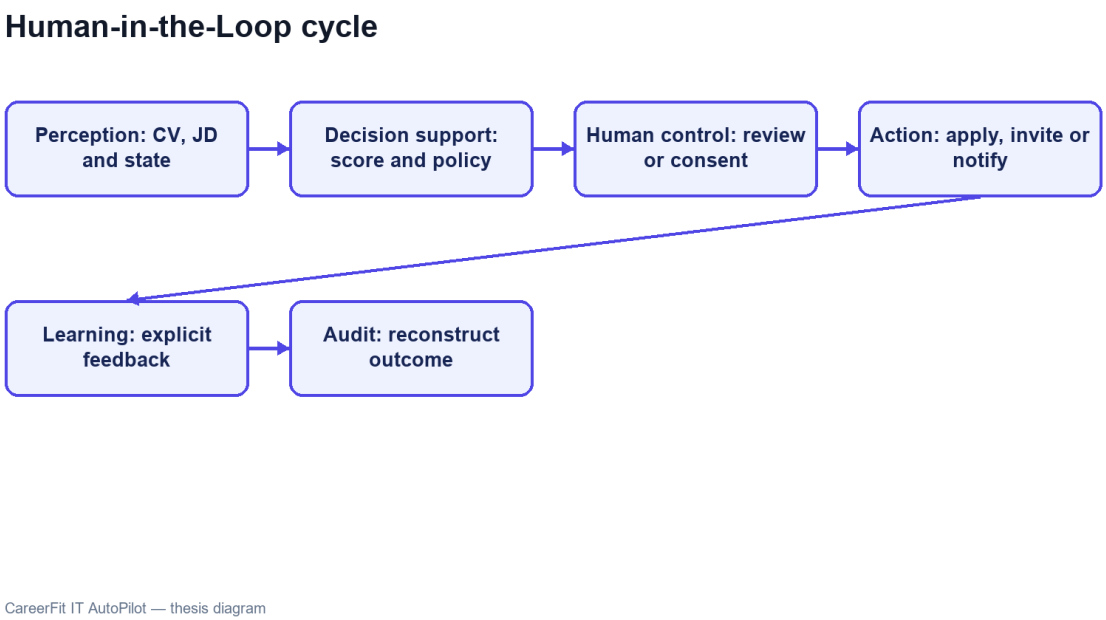

Figure 2.4. Human-in-the-Loop control cycle in CareerFit

2.8 Web Architecture, Security, and Audit Foundations

2.8.1 Client–Server and REST

CareerFit uses a web client and a server-side application connected through HTTP APIs. The client is responsible for presentation, navigation, local interaction state, and invoking authorized operations. The backend enforces authentication, authorization, validation, business rules, transactions, persistence, and background processing. This separation prevents browser-side presentation logic from becoming the authority for protected actions.

REST is an architectural style characterized by constraints including client–server separation, stateless interaction, cacheability where appropriate, a uniform interface, layered components, and optional code-on-demand [11]. A REST API models resources and transfers representations of their state. In practice, merely using JSON over HTTP does not guarantee that every REST constraint is satisfied; Chapter 3 therefore describes CareerFit as a REST-oriented API and documents its actual endpoint and state-transition behavior.

2.8.2 Authentication, Authorization, and Action Tokens

Authentication establishes the identity associated with a request, while authorization determines whether that identity may perform an operation. Role-Based Access Control assigns permissions according to roles such as Candidate, Recruiter, or Administrator, but ownership checks are still required; for example, one candidate must not access another candidate's private CV merely because both share the same role.

JSON Web Token is a compact, URL-safe format for transferring claims that can be signed or message-authentication-code protected and may include registered claims such as subject, audience, expiration time, issued-at time, and token identifier [12]. A valid signature alone is not sufficient: the application must validate the expected algorithm and relevant claims and must apply its own account and authorization rules. Short-lived access tokens and purpose-specific action tokens address different workflows and should not be treated as interchangeable credentials.

An actionable email link may be opened by the intended user, forwarded by mistake, or visited by a mail-security scanner. CareerFit uses a two-step flow to reduce this risk. The GET request checks the high-entropy token and shows a confirmation page without changing recruitment data. A deliberate POST request performs the action and marks the token as redeemed. Only the SHA-256 token hash is stored, and status and expiry checks prevent normal reuse. Production deployment should still add rate limiting, origin controls, secret rotation, and mail-delivery monitoring.

2.8.3 Audit Logging

Audit logging records security-relevant and business-relevant events so that operations can be monitored and investigated. NIST guidance emphasizes a managed logging process that includes generation, transmission, storage, access, analysis, and disposal rather than treating logs as incidental text files [13]. For recruitment automation, useful fields include the actor, action, target, source channel, relevant policy or score snapshot, result, and timestamp. Sensitive CV content and credentials should not be copied into logs without a clear need.

An append-oriented audit history supports traceability, but database permissions, retention, integrity, and access control determine whether that history is trustworthy. Operational logs used for debugging and domain audit records used to explain a business action may have different schemas and retention requirements. Chapter 2 defines these responsibilities at the design level, and Chapter 3 describes the mechanisms actually implemented in CareerFit.

2.9 Related Work and Research Gap

Prior work can be grouped into lexical retrieval, learned resume–job representation, explainable recommendation, and human-centered governance. The classical vector-space and relevance-feedback literature provides transparent mathematical foundations [1], [4]. Contemporary resume–job matching methods such as ConFit v2 use trained dense representations and hard-negative strategies to improve semantic matching [7]. Real-world observational research comparing human and LLM resume ratings found only minor correlation between the two, indicating that automated and human judgments should not be treated as interchangeable [8]. Explainable recommendation research seeks to accompany scores with understandable, preferably grounded reasons [14].

CareerFit does not attempt to outperform these research systems on a shared benchmark. Its engineering gap is different: connecting an inspectable lexical baseline and explicit relevance feedback to an end-to-end recruitment workflow with role-based interfaces, policy gating, actionable email, and auditable state changes. The resulting contribution is system integration and controlled automation within an IT-focused academic prototype.

Table 2.1. Positioning of CareerFit against major approach categories

Approach	Strength	Limitation relative to CareerFit scope	CareerFit position
Lexical TF-IDF/cosine	Transparent, efficient, reproducible	Limited semantic equivalence	Implemented baseline with normalization, reasons, and validation
Dense resume–job encoders	Captures semantic relations	Requires training data, governance, and benchmark validation	Future hybrid/semantic extension; not claimed as implemented

Approach	Strength	Limitation relative to CareerFit scope	CareerFit position
General job portal	Strong search, publication, and application UX	May not expose scoring, feedback, or policy internals	Combines portal workflows with inspectable matching and automation
Fully automated decision pipeline	Reduces manual steps	Higher control, accountability, and error-propagation risk	Policy-gated HITL actions with confirmation and audit
Explainable recommender	Provides user-facing reasons	Explanation may be unfaithful if not grounded	Uses reasons tied to processed fields and lexical evidence

The comparison shows that CareerFit deliberately accepts the semantic limitations of a lexical baseline in exchange for inspectability and implementation control. Its research value must therefore be assessed through reproducible behavior, workflow integration, and clearly stated limitations rather than through unsupported claims of general AI superiority.

2.10 Theoretical Background Summary

This chapter presented the main ideas used by CareerFit. TF-IDF and cosine similarity provide an inspectable lexical baseline, while Rocchio uses explicit feedback to adjust ranking. Ranking metrics measure ordered results, and Human-in-the-Loop policies keep similarity evidence separate from recruitment actions. Chapters 1 and 2 turn these ideas into system requirements and design decisions.

2.11 System Architecture

CareerFit is implemented as a modular monolith. The frontend is a separate React application, while the backend is one Spring Boot deployable application organized into domain packages. This is not a microservice architecture: modules share one process, one primary PostgreSQL database, common security and exception handling, and direct in-process service calls. The modular monolith reduces deployment and distributed-transaction complexity while preserving boundaries that can support later extraction if scale or ownership requires it.

The presentation layer contains React pages, reusable components, route guards, API mapping, and query state. It calls the backend over HTTP and must not be trusted to enforce protected business rules. The API layer contains controllers and DTO validation. The service layer owns use-case orchestration, transactions, scoring, policy decisions, and ownership checks. Repositories map domain entities to PostgreSQL through Spring Data JPA. Cross-cutting configuration covers security, CORS, async execution, exception responses, scheduling, API documentation, and health/metrics endpoints.

The backend stores relational business state in PostgreSQL. JSONB is used for naturally variable collections or snapshots such as skills, extracted terms, vectors, reasons, benefits, policy metadata, and analytic distributions. CV binaries are stored through a storage service in a configured local path or Docker volume; the database

stores file metadata and the processing result. SMTP is optional, and a no-operation mail implementation allows local workflows when outbound email is disabled.

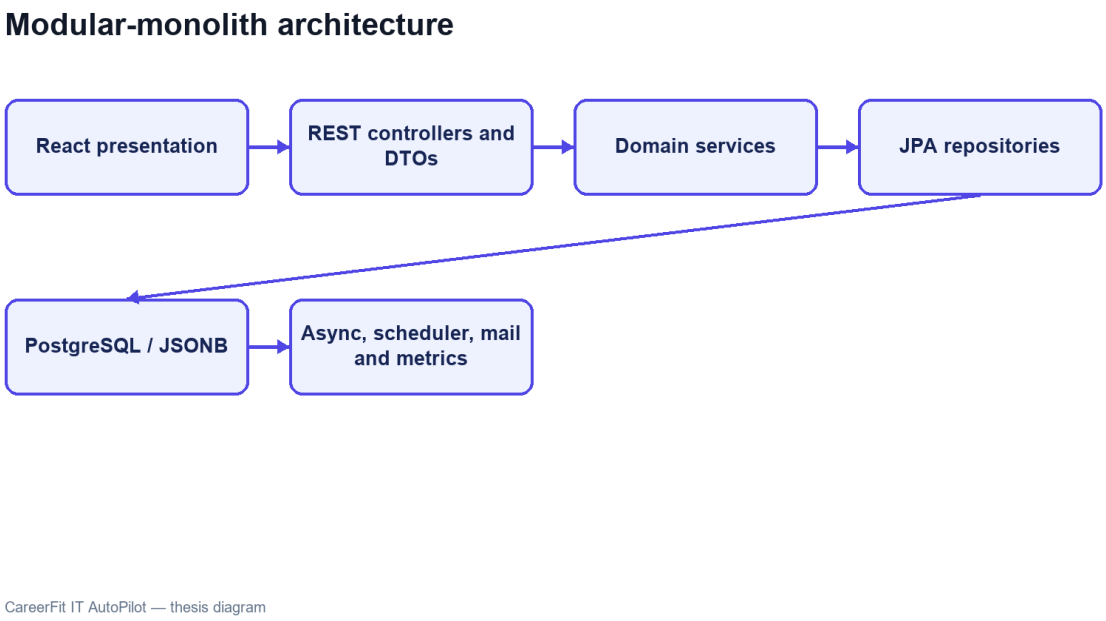

Figure 2.5. CareerFit container and component architecture

2.12 Module Design

Table 2.2. Backend modules and responsibilities

Module	Main responsibility	Representative services
Auth and Security	Registration, login, passwordless flow, JWT validation, account resolution, role enforcement	AuthService, JwtService, security filters
Candidate and CV	Candidate profile, portfolio, CV lifecycle, storage, extraction/OCR, validation	CandidateProfileService, CvManagementService, CvIngestionService, PdfExtractionService
Job and Employer	Public job search, recruiter-owned job lifecycle, employer pages and profile	JobService, EmployerService
Matching	TF-IDF/cosine scoring, labels, reasons, batch matching, candidate cards and recruiter ranking	TfIdfService, ScoringService, MatchingService, MatchingBatchService, MatchingQueryService
Recommendation	Candidate-profile job ordering and similar-job retrieval	RecommendationService
Application	Manual applications, withdrawal, applicants, invitation, recruiter status transitions	ApplicationService
Feedback	Feedback validation, persistence, Rocchio update, recomputation signaling	FeedbackService, RocchioService
Automation	Policy persistence, run-now, auto-apply, pause/resume, scheduled orchestration	AutomationPolicyService, AutoApplyService, AutomationScheduler
Notification	Policy guard, delivery log, SMTP/no-op sending, actionable email	NotificationPolicyGuard, NotificationEmailService, EmailActionService

Module	Main responsibility	Representative services
Analytics and Admin	Market/role analytics, events, administrative dashboards, moderation and monitoring	AnalyticsService, AdvancedAnalyticsService, administrative services
Audit and Common	Audit persistence, shared responses, exceptions, validation utilities	Audit repository/service and common components

The modules cooperate through service calls rather than direct controller-to-repository shortcuts for core workflows. Transaction boundaries belong at service operations that change a consistent group of records. Read-only queries may use dedicated query services or projections to avoid loading unrelated state. DTOs prevent persistence entities from becoming an accidental public API contract.

2.13 Data Design

2.13.1 Core Identity and Recruitment Data

user_account stores identity, role, activation, verification, language, and password hash. A Candidate account owns one candidate profile, multiple CVs, portfolio links, and portfolio project records. A Recruiter account owns an employer_profile and jobs through recruiter_id. job stores the original JD, structured recruitment attributes, salary mode, language, status, source provenance, and vector data.

matching is the unique association between one CV and one job. It stores raw and normalized scores, label, potential metadata, reasons, and the recomputation marker. application links Candidate and Job and may reference the CV and Matching used at application time. feedback links a Matching with its actor and source channel. recommendation_interaction records behavioral events such as viewed, skipped, applied, saved, not interested, or show similar.

2.13.2 Automation, Communication, and Operational Data

automation_policy stores one policy per user. email_action and email_token currently represent two distinct tokenized workflows. Notification delivery records support deduplication and quota/cooldown decisions. audit_log stores actor, action, target, result, source channel, request metadata, and structured metadata. Job trend, market snapshot, and analytics event tables support aggregate dashboards without changing the transactional meaning of matching records.

Table 2.3. Key integrity constraints

Constraint	Purpose
Unique normalized user email	Prevent duplicate identities that differ only by case
One Candidate per user and one policy per user	Preserve one-to-one domain ownership
Partial unique default CV per Candidate	Ensure at most one active default CV
Unique Matching per CV and Job	Prevent duplicate scores for the same pair
Unique Application per Candidate and Job	Prevent duplicate applications
Unique feedback per Matching and actor	Prevent ambiguous duplicate current judgments
Check constraints on roles, statuses, labels, salary mode, and source channels	Reject invalid domain states at the database boundary
Optimistic version columns on mutable core entities	Detect conflicting concurrent updates
Source hash index on imported jobs	Support idempotent scraped-job upsert

Flyway migrations are the authoritative schema history. Hibernate uses ddl-auto=validate, so the application validates entity-to-schema compatibility instead of silently generating a different database. Indexes support common access paths such as active jobs, recruiter jobs, CV ownership, matching by score, applications by candidate or job, feedback history, tokens, and audit chronology.

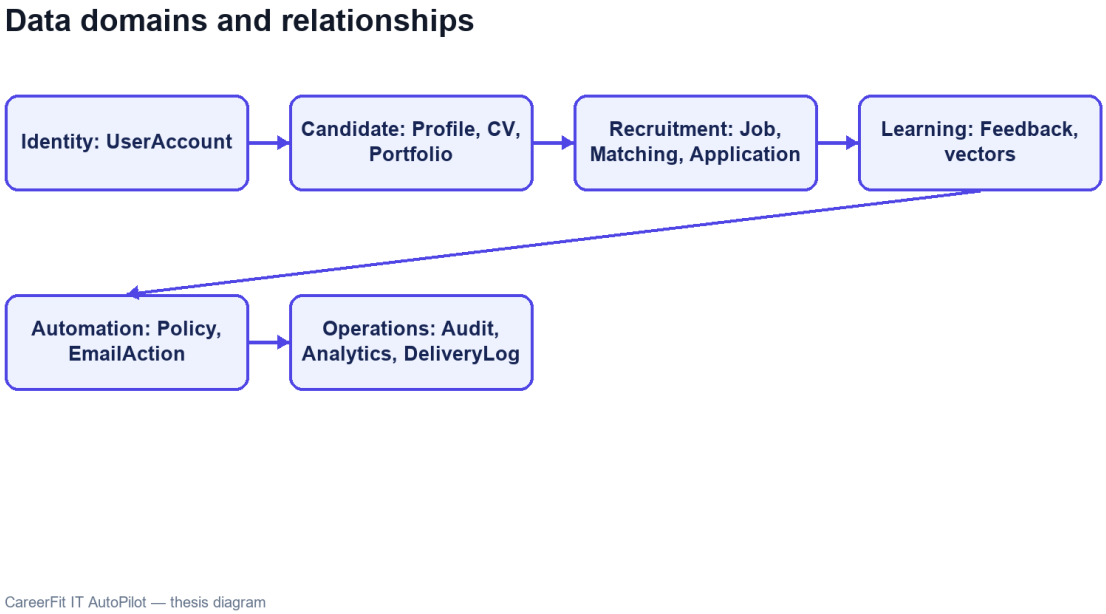

Figure 2.6. CareerFit logical entity relationships

2.14 Security, Failure, and Consistency Design

2.14.1 Authentication and Authorization

The backend is stateless at the HTTP session layer. A JWT authentication filter runs before username/password authentication processing, and a user-ID resolution filter runs after JWT validation. Public access is explicitly limited to authentication entry points, selected job/employer/analytics GET endpoints, health/metrics/API documentation, and email-action redemption. Candidate, Recruiter, and Administrator endpoint groups require corresponding roles. Automation requires authentication, with service logic responsible for policy ownership.

The security configuration disables CSRF because the application uses stateless bearer authentication, configures a CORS origin allow-list, denies framing, applies a content-security policy, and configures HSTS. These headers do not replace TLS termination in the deployed environment. Public Swagger and Prometheus access are acceptable for local development but should be restricted in a production deployment.

2.14.2 Failure Handling

CV processing represents long-running work with statuses rather than holding a browser request open until every score completes. Extraction or validation failure records a failure state and reason. CV and Job matching tasks are submitted through an after-commit executor so background work cannot read uncommitted rows. Batch scoring isolates individual pair failures, and scheduled recomputation keeps unsuccessful rows marked for later review instead of falsely marking them complete.

Email can operate through SMTP or a no-op implementation. Notification delivery outcomes are recorded as sent, skipped, or failed, and policy guards enforce deduplication, quota, cooldown, and timing rules. Email failure must not roll back an already valid matching result. Conversely, application creation and status changes use transactions and uniqueness constraints because partial persistence would change recruitment state.

2.14.3 Identified Design Risks

Table 2.4. Design risks and required treatment

Risk	Current control	Remaining treatment
Unauthorized cross-account access	URL role rules and service ownership checks	Maintain negative integration tests for every identifier-based operation
Duplicate applications or matchings	Service checks and unique constraints	Convert database conflicts into stable API errors
Replayed email action	Pending/redeemed state and expiry	Implemented hashed storage, GET confirmation, and POST execution; retain replay and expiry tests
Link scanner triggers action	GET is non-mutating and displays confirmation	Add rate limiting and deployment-specific origin controls
Scheduler/configuration drift	Scheduler timing reads from app.scheduler properties	Maintain configuration and schedule-boundary tests
Sensitive data in logs	Structured audit metadata and DTO boundaries	Review retention, redaction, access, and exported evidence
Lexical scoring bias or omission	Reasons, validation, feedback, and HITL	Evaluate representative data; never treat score as hiring probability
Local file-storage loss or exposure	Configured path and Docker volume	Define backup, encryption, malware scanning, and retention before production

2.15 Deployment Architecture

The development deployment uses Docker Compose for PostgreSQL and optionally the backend. PostgreSQL 16 exposes container port 5432 as host port 5433 by default. A backend process running directly on the host connects to localhost:5433; the backend container connects through the Compose network to postgres:5432. Confusing these two addresses is a common configuration failure.

The optional backend container exposes port 8080, depends on PostgreSQL health, and mounts a named volume for CV storage. The React/Vite frontend is normally run separately on port 5173 during development and calls the backend API. Environment

variables override database credentials, JWT secret, application base URL, CORS origins, mail settings, OCR executable and languages, and storage path. Tesseract with Vietnamese and English language data is included in the backend container path described by the project Docker configuration.

Actuator exposes health and Prometheus metrics according to the current security configuration. PostgreSQL health checks gate backend container startup. This architecture is appropriate for reproducible local development and demonstration, but it is not by itself a production topology. A production design would additionally require managed secrets, TLS termination, restricted management endpoints, database backup and recovery, protected object storage, centralized logs, monitoring alerts, and explicit scaling and retention policies.

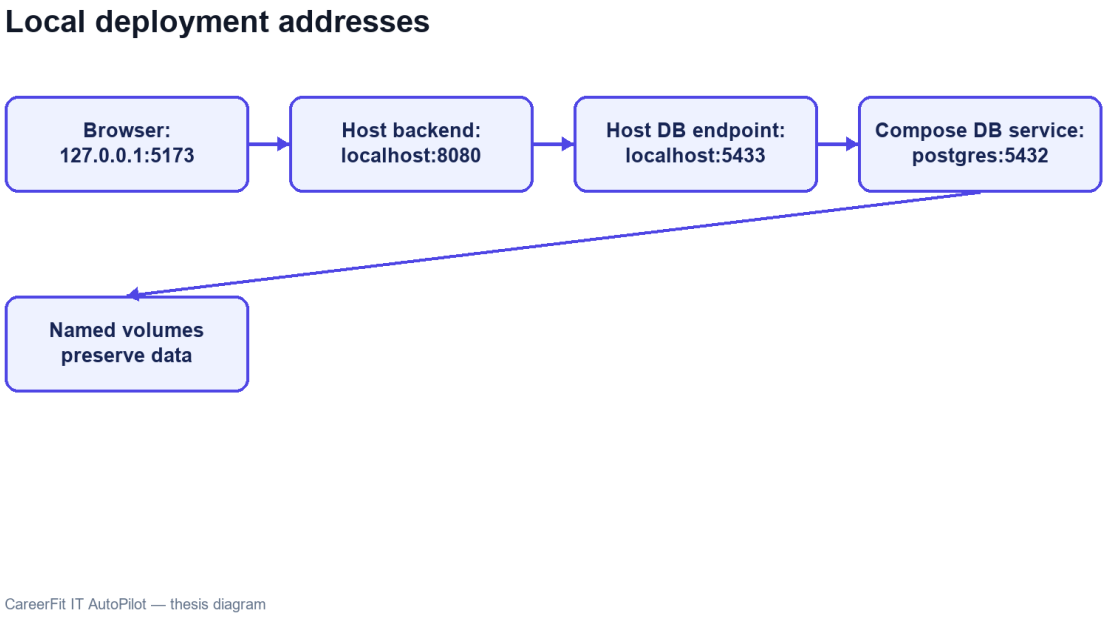

Figure 2.7. Local and containerized deployment topology

2.16 Chapter Summary

This chapter connected the theoretical foundations to the CareerFit solution design. TF-IDF, cosine similarity, Rocchio feedback, ranking metrics, and Human-in-the-Loop principles explain the core decision-support behavior. The architecture, modules, data model, security boundaries, consistency rules, and deployment topology define how those ideas are separated into implementable components.